EvidenceGate: Checking Security Evidence Against CI/CD Release Artifacts

A Controlled Synthetic Evaluation and Software Testing Study

Abstract

CI/CD pipelines may collect security records that describe different releases. This study presents EvidenceGate, a prototype that compares build provenance, a software bill of materials (SBOM), and vulnerability evidence with an independently selected artifact digest. Ten explainable checks assess identity consistency, declared source and builder, scan freshness, and vulnerability policy.

Evaluation used 27 functional fixtures, a separate 64-case factorial suite, 31 automated software tests, and 10 browser scenarios. On the functional suite, presence checks blocked 3 of 22 faulty cases, an ablation without artifact binding blocked 17, and the full checker blocked 22. The corresponding factorial counts were 0, 56, and 63 of 63 faulty combinations.

All software tests and browser scenarios passed. Across 400 timing observations, median in-memory parsing plus evaluation ranged from 0.028 to 17.516 ms. These results establish behavior on constructed inputs. They do not establish field effectiveness, cryptographic authenticity, scanner accuracy, or research novelty. Consistent but false declarations remain outside the prototype’s detection scope.

Keywords: DevSecOps; CI/CD; software supply chain; artifact identity; SBOM; provenance; policy checking; synthetic evaluation.

1. Introduction

Security evidence is useful only when it concerns the intended release. A recent vulnerability report may describe an earlier artifact, while an SBOM may use a different identity. Checking each document separately can miss this relationship.

EvidenceGate addresses this specific consistency problem. It compares three evidence records with an externally selected SHA-256 and applies a release policy. The study evaluates the implemented checker rather than proposing a new cryptographic protocol or claiming that document agreement proves software safety.

The study asks three questions:

1. **RQ1:** What additional faulty cases are blocked when artifact-binding checks are enabled?
2. **RQ2:** How does processing time change with larger SBOM and vulnerability inputs?
3. **RQ3:** Which implementation behaviors pass testing, and which trust failures remain unobservable?

The contributions are an executable prototype, traceable synthetic comparisons, measured processing costs, and documented software tests. All reported measurements were collected from the existing implementation. The two synthetic datasets are analyzed separately because they share construction assumptions.

2. Related Approaches and Scope

Build provenance, component inventories, vulnerability reports, and signature verification answer different questions. SLSA and in-toto inform production statements; CycloneDX structures inventory information; a scanner such as Trivy supplies vulnerability findings. Signature verification can authenticate signed records when its trust policy is correctly applied.

EvidenceGate consumes supported claims and checks their agreement with a selected release. Artifact identity checks are an established principle. This study evaluates one explicit implementation and its ablation; it does not establish that existing verification systems lack similar capabilities.

Table 1. Roles of related approaches and their relationship to this study.

Approach	Primary role	Use in this study	Evaluation boundary
SLSA / in-toto	Describe production and process claims	Supported raw provenance input	Structural and policy checks; no signature verification
CycloneDX	Represent component inventories	SBOM input with root artifact hash	Supported subset, not full schema conformance
Trivy	Report vulnerability findings	Optional native-report adapter	Adapter contract tests; no live scanner run
Signature verification	Authenticate signed statements under a trust policy	External prerequisite for authentic evidence	Not implemented or benchmarked here
EvidenceGate	Check joint evidence consistency and release policy	Prototype under evaluation	Synthetic decisions, timing, and interface tests

The measured comparators are internal decision strategies. Table 1 is a functional comparison, not a measured ranking of these technologies. No superiority over a configured external policy engine is claimed.

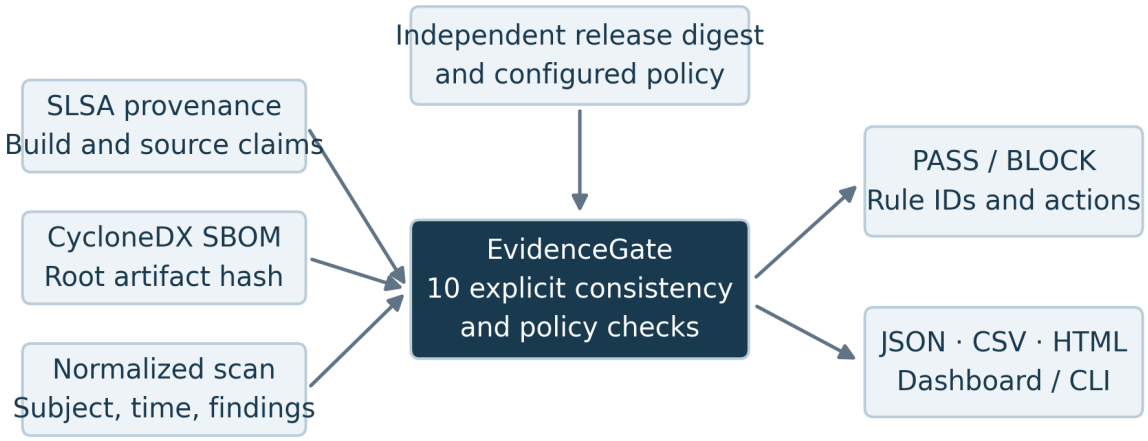
3. Prototype Design

3.1 Inputs and trust boundary

The expected artifact digest and policy must be selected independently of the supplied evidence. Inputs comprise a raw in-toto Statement v1 with SLSA provenance v1, CycloneDX JSON 1.5 or 1.6, and a normalized vulnerability record.

For an OCI image, the chosen identity should be the manifest digest. An image configuration ID and the hash of an exported archive identify different bytes. The Trivy adapter therefore matches the selected digest against `Metadata.RepoDigests` and rejects an `ImageID` substitution.

Figure 1. Inputs, rule evaluation, and outputs of the EvidenceGate prototype.



Evidence authenticity and scanner correctness must be established externally.

The tool checks visible declarations. A matching builder identifier is an allowlisted claim, not authenticated producer identity. Timestamp accuracy, scanner completeness, and evidence authenticity require a trusted collection process outside the checker.

3.2 Acceptance conditions

Let d be the selected release digest. Acceptance requires d in the provenance subject set, the SBOM root SHA-256 equal to d, and the scan subject SHA-256 equal to d. It also requires all source, builder, structure, age, and severity conditions to pass.

Table 2. Implemented checks and their rejection conditions.

Rule	Checked property	Rejection condition
EG001	Provenance structure	Missing or unsupported statement
EG002	Provenance artifact	Selected release digest absent
EG003	Declared builder	Builder absent or not allowlisted
EG004	Declared source	Expected URI/commit absent, different, or conflicting
EG005	SBOM structure	Missing or unsupported inventory
EG006	SBOM root artifact	SHA-256 absent, different, or conflicting
EG007	Scan structure	Missing, malformed, or declared partial scan
EG008	Scan artifact	Subject digest differs from selected release
EG009	Scan timestamp	Invalid, stale, or implausibly future time
EG010	Vulnerability policy	Blocked severity present or scan invalid

The evaluated policy allows one fictional builder and one full source commit. It blocks HIGH and CRITICAL findings and limits scan age to 72 hours. A future timestamp is tolerated only within five minutes. These are experimental settings, not universal security recommendations.

3.3 Decision and reporting

Every failed rule produces BLOCK. Unreadable input, invalid JSON, and invalid policy produce a nonzero input-error exit. An omitted record is never interpreted as a clean assessment. Reports identify failed rules and remediation actions.

The command-line and local browser interfaces use the same evaluator. JSON, CSV, and HTML exports support inspection and automation. The implementation uses Python's standard library at runtime; publication figures and document generation use separate research dependencies.

4. Evaluation Method

4.1 Datasets and labels

F27 reproduces the existing 27-case suite: five valid controls and 22 constructed faults. Fault categories cover missing records, unsupported structures, artifact binding, builder/source policy, scan dates, and blocking vulnerability findings. Labels follow the documented mutation rather than an incident investigation.

A faulty case violates this experiment's acceptance contract. It does not necessarily contain exploitable software; for example, an unsupported SBOM version is rejected because the input is outside the implemented contract.

F64 enumerates every combination of six Boolean factors: provenance mismatch, SBOM mismatch, scan mismatch, unapproved builder, stale scan, and a critical finding. The all-false case is valid; the other 63 combinations should be blocked under the experimental policy.

Table 3. Evaluation components and what they measure.

Component	Size	Construction	Interpretation
F27 functional suite	27 cases: 5 valid / 22 faulty	Existing seeded fixtures; seed 7	Known rule behavior
F64 factorial suite	64 cases: 1 valid / 63 faulty	All combinations of six fault flags	Interactions and masking by other faults
Residual trust controls	2 demonstrations	False external assumptions with consistent visible inputs	Unobservable conditions; excluded from metrics
Software tests	31 unittest tests	Core, adapter, CLI, and HTTP checks	Implementation behavior, not attack coverage
Browser scenarios	10 checks	Headless Chromium; desktop and mobile	Interface behavior, not usability research
Timing experiment	400 retained observations	4 input sizes × 100 repetitions	In-memory parsing and checking cost

The suites reuse synthetic templates and contain overlapping mechanisms. Their counts are not pooled into one accuracy claim. No real repositories, organizations, naturally occurring incidents, or independently labeled security dataset were sampled.

4.2 Comparison modes

Presence checks whether provenance, SBOM, and scan documents are non-null. This deliberately weak baseline evaluates availability, not content.

Ablation uses the full rule outcomes but excludes EG002, EG006, and EG008 from its decision. This retains seven checks and isolates artifact binding. It is a decision ablation, not a separately optimized program.

Full applies all ten EvidenceGate rules. The comparisons concern decisions only. Because the ablation reuses the full evaluator's outcomes, no timing comparison between these modes is reported.

4.3 Metrics

A faulty case is the positive class. TP denotes a faulty case blocked; FN denotes a faulty case accepted. FP denotes a valid case blocked; TN denotes a valid case accepted. Recall is TP / (TP + FN), and F1 is 2TP / (2TP + FP + FN).

Precision is undefined when a mode blocks no cases. Constructed examples do not justify population confidence intervals or statistical significance claims. Counts and fault mechanisms are therefore emphasized alongside descriptive metrics.

4.4 Timing procedure and environment

Timing used Python 3.12.14 on Linux x86-64 in a shared cloud environment reporting an AMD EPYC 9V74 80-Core Processor. The process had affinity to 5 logical CPUs. No dedicated CPU isolation, production load control, or memory profiling was performed.

Inputs contained 0, 100, 1,000, or 10,000 SBOM components and the same number of LOW-severity scan findings. Ten warm-up iterations preceded 100 measured iterations at each size. Each observation recorded strict JSON parsing, full rule evaluation, and their combined interval using `perf_counter_ns`.

Input generation and serialization occurred before timing. Disk I/O, artifact hashing, scanner execution, signature verification, report serialization, network transfer, and deployment were excluded. Medians and interquartile ranges describe this one run; intervals are not confidence bounds.

4.5 Software and browser testing

The existing unittest suite was rerun without modifying the tool. It covers normal decisions, rejection conditions, time boundaries, conflicting digests, malformed nested values, JSON handling, report escaping, adapter rules, command exits, and local HTTP restrictions.

Browser automation used Chromium 154.0.8037.57 at 1440-pixel and 390-pixel viewport widths. It exercised four decision scenarios, filtering, two downloads, duplicate-key rejection, file hashing, and the mobile layout. This is automated interface validation, not a participant study.

5. Results

5.1 Functional decisions

On F27, the full checker blocked all 22 constructed faults and accepted all five valid controls. The ablation blocked 17 faults and accepted five faulty cases. Presence checks blocked only the three cases with missing documents. Figure 2 compares blocking rates; Table 4 reports the underlying counts.

Figure 2. Blocking of faulty cases in F27 and F64; labels give counts and within-suite percentages.

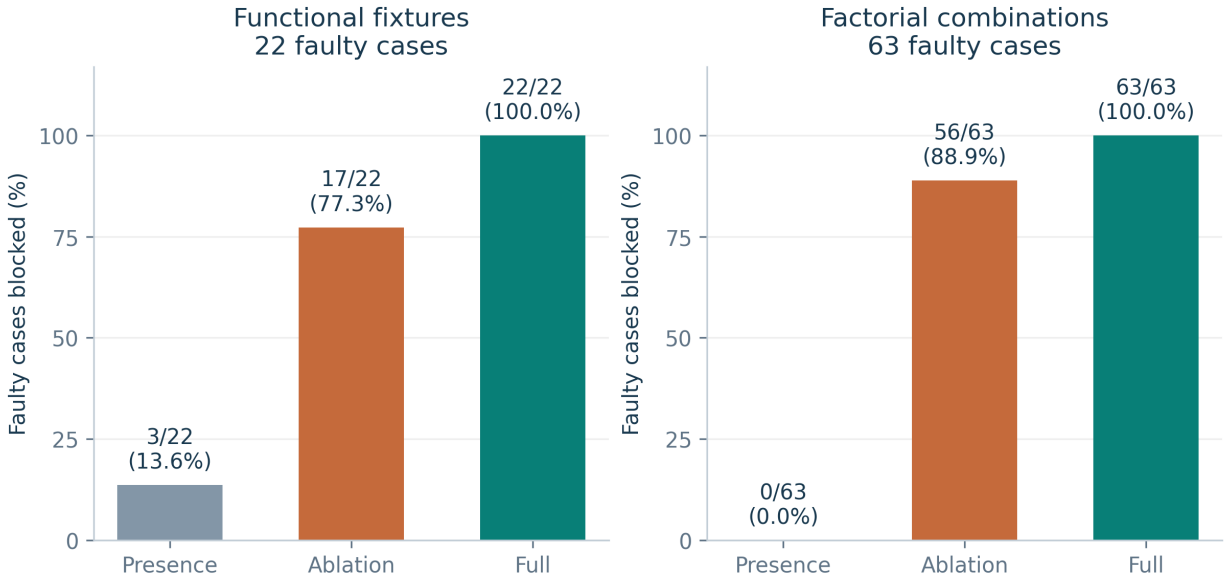
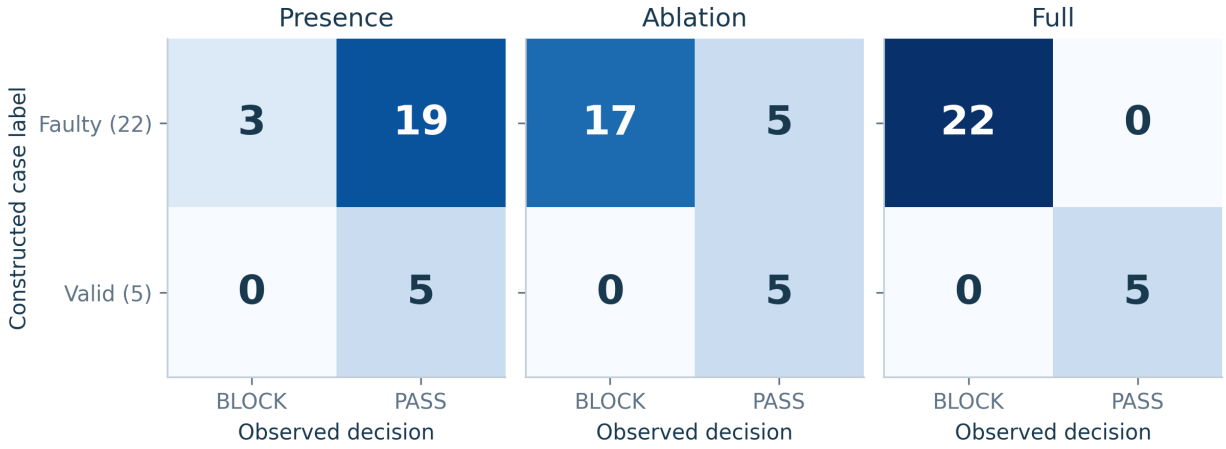


Table 4. Observed confusion counts and descriptive metrics; F27 and F64 are analyzed separately.

Set	Mode	TP	FN	FP	TN	Recall	F1
F27	Presence	3	19	0	5	13.6%	0.240
F27	Ablation	17	5	0	5	77.3%	0.872
F27	Full	22	0	0	5	100.0%	1.000
F64	Presence	0	63	0	1	0.0%	0.000
F64	Ablation	56	7	0	1	88.9%	0.941
F64	Full	63	0	0	1	100.0%	1.000

F27 recall increased from 77.3% in the ablation to 100.0% with binding, a 22.7 percentage-point difference on these fixtures. All three modes had precision 1.000 because none blocked the five valid controls. This small constructed control set does not establish a low operational false-block rate.

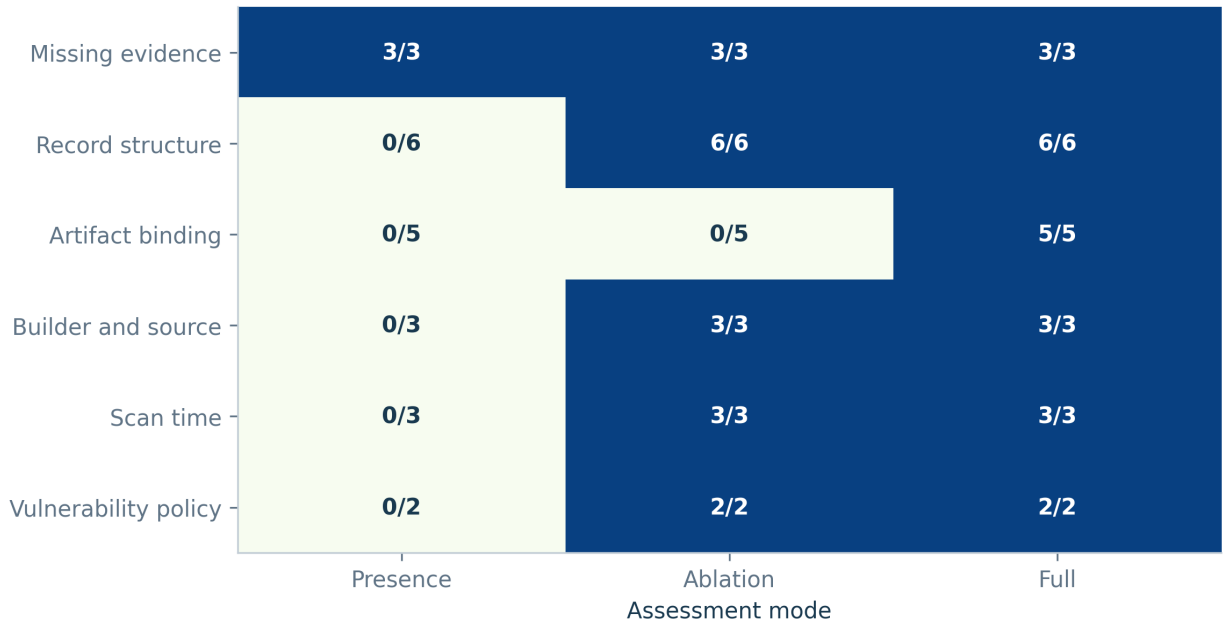
Figure 3. Confusion matrices for F27; rows are constructed labels and columns are observed decisions.



5.2 Which faults explain the difference?

The five additional F27 blocks concern reused provenance, a reused SBOM, a reused scan, an absent root hash, and conflicting root hashes. These cases satisfy the retained structure and policy conditions but fail at least one removed binding rule.

Figure 4. F27 faulty cases blocked within each category; cell labels are blocked / total.



The category analysis explains the aggregate result. The benefit comes from testing evidence identity against the selected release, rather than adding a different vulnerability detector. No new vulnerabilities were discovered by this experiment.

5.3 Factorial interactions

In F64, the full checker blocked all 63 faulty combinations. The ablation blocked 56 and accepted seven; those seven contain one or more binding faults with no unapproved builder, stale date, or critical finding. Presence checks accepted every combination because all documents remained present.

Other policy violations mask missing binding checks in many combinations: the ablation still blocks a reused scan when the same case contains a critical finding. Its 88.9% recall therefore does not mean artifact identity is adequately checked. The full suite's higher score reflects the stated Boolean construction.

No mode blocked the single valid F64 combination. Precision for Presence is undefined because it produced no blocks. The overlapping synthetic mechanisms and extreme class balance prevent interpreting F64 as an estimate of deployment prevalence or field accuracy.

5.4 Processing cost

Median parsing plus evaluation was 0.028 ms for the 1,462-byte input and 17.516 ms for the 1,269,240-byte input. At the largest size, median parsing was 11.873 ms and evaluation was 5.590 ms. These are separately computed medians and need not sum exactly.

Figure 5. Measured processing time across four input sizes; points are medians and whiskers are interquartile ranges.

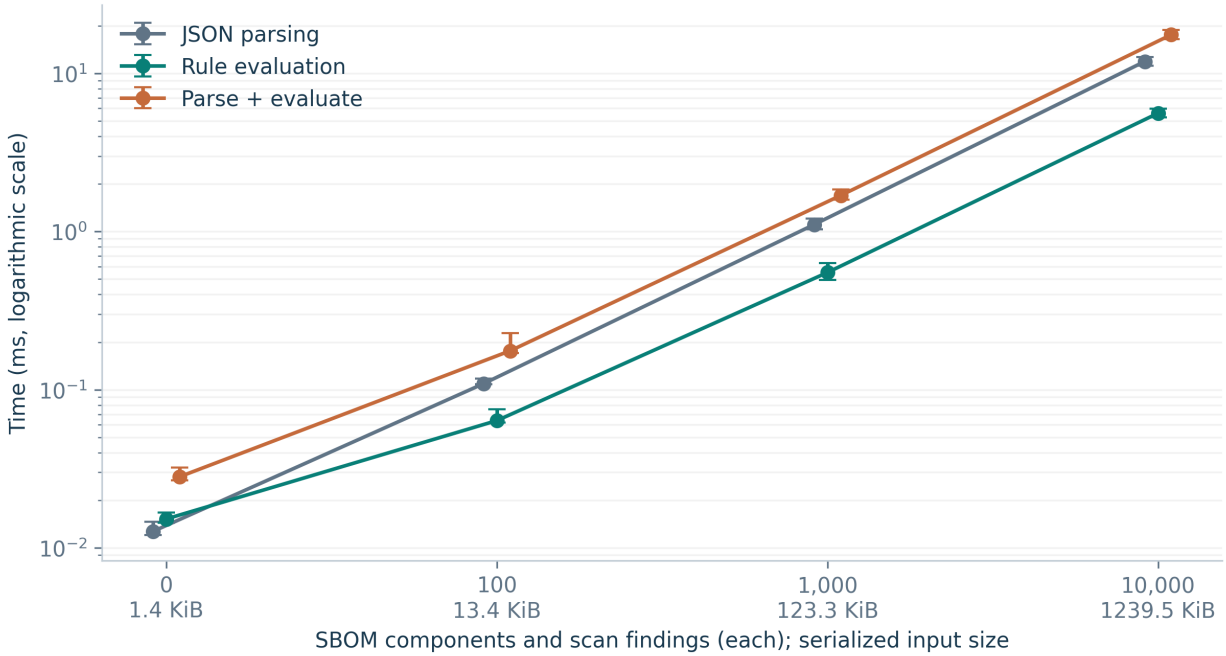


Table 5. Processing-time measurements: 100 retained repetitions and 10 warm-ups per size.

Records each	Input KiB	Parse ms	Check ms	Combined ms	Combined IQR ms
0	1.4	0.013	0.015	0.028	0.027–0.032
100	13.4	0.109	0.064	0.176	0.171–0.227
1,000	123.3	1.106	0.550	1.688	1.586–1.844
10,000	1239.5	11.873	5.590	17.516	16.408–18.722

The largest input contains 10,000 components plus 10,000 findings, not 10,000 records in total. Processing increased with this workload. Four sizes in one shared environment are insufficient to establish an asymptotic model, throughput guarantee, or production pipeline overhead.

5.5 Test outcomes

All 31 unittest tests passed, with no failures, errors, or skipped tests. All 10 browser scenarios passed, and no JavaScript page errors were observed. Figure 6 and Table 6 summarize the executed checks; individual test names and logs are retained with the paper.

Figure 6. Executed software tests and browser scenarios by group; all passed.

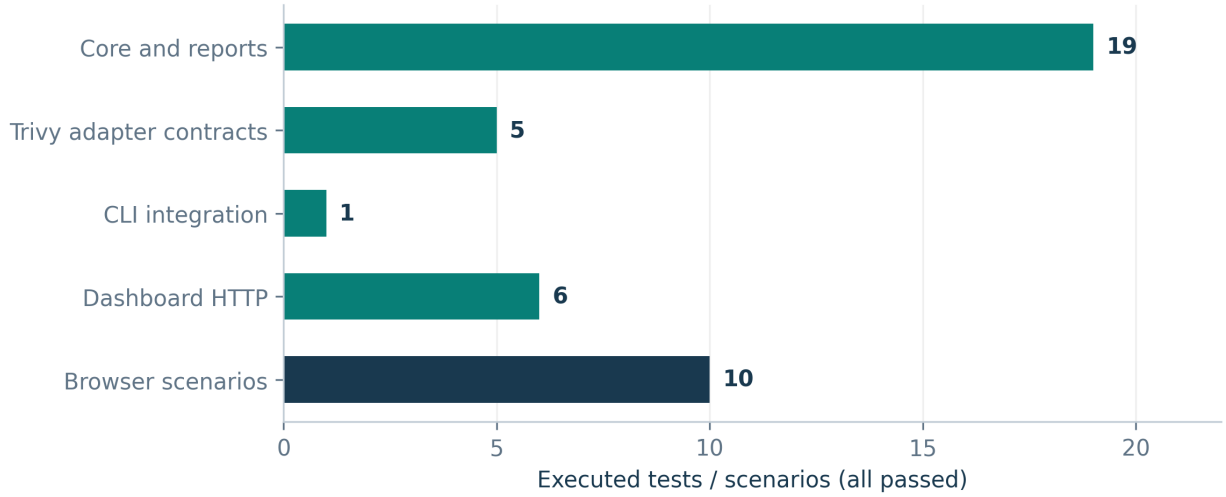


Table 6. Software-validation results and their practical boundaries.

Test group	Count	Outcome	What was exercised
Core and reports	19	19 passed	Rules, malformed data, time boundaries, escaping, benchmark contract
Trivy adapter	5	5 passed	Manifest binding, missing results, severity preservation; fixtures only
CLI integration	1	1 passed	PASS, BLOCK, and input-error exit codes within one test
Dashboard HTTP	6	6 passed	API evaluation, origin/host checks, duplicate keys, path/media handling
Browser scenarios	10	10 passed	Four decisions, filter, two exports, invalid upload, hashing, mobile layout

A test function can contain several assertions or subtests. These execution counts are not percentages of attack coverage, code coverage, or production reliability. The Trivy adapter was tested against constructed JSON; no live scanner run was performed.

5.6 Residual trust controls

Two additional demonstrations kept all visible inputs consistent while breaking an external assumption. Both produced PASS with ten passing checks. They are excluded from confusion metrics because the violated condition is not available to the checker.

Table 7. Observed behavior when an external trust assumption is false.

Control	Visible evidence	Observed result	Implication
Unauthenticated producer claim	Matching fictional builder/source and artifact declarations	PASS; 10/10 checks	Agreement cannot authenticate the producer
Omitted critical finding	Clean scan record; an out-of-band synthetic label says a finding was omitted	PASS; 10/10 checks	The checker cannot discover findings absent from its input

These controls demonstrate an observability boundary, not a successful real-world exploit or an empirical scanner false-negative rate. They explain why a passing policy report must not be presented as proof of safe software.

6. Discussion

RQ1: Artifact binding rejected five additional isolated F27 faults and seven F64 combinations that passed the ablation. The mechanism is explicit comparison with an independently chosen digest. A separate vulnerability threshold cannot replace that comparison.

RQ2: In-memory processing increased from approximately 0.028 to 17.516 ms across the tested sizes. Parsing accounted for more time than rule evaluation at the largest size. These measurements characterize local record processing rather than the cost of collecting or authenticating evidence.

RQ3: The observed software and browser tests passed, but authenticity and hidden scanner errors remained unobservable. Deployment should therefore combine consistency checking with trusted collection, external signature verification, and enforcement of the intended release digest.

The practical lesson is to distinguish availability, policy compliance, identity agreement, and authenticity. A pipeline can satisfy some of these conditions while failing others. Clear rule outcomes make those distinctions inspectable without creating a combined security score.

7. Limitations and Reproducibility

7.1 Threats to validity

- **Constructed labels:** Faults were selected to exercise known rules. High within-suite scores are expected and are not independent validation.
- **Limited controls:** F27 contains five valid cases; F64 contains one. Neither estimates the operational false-block rate.
- **Related datasets:** Both suites reuse the same template and assumptions. Their observations are not independent samples of real pipelines.
- **Comparator scope:** Presence is deliberately weak; the ablation reuses full-engine outcomes. No configured external product was benchmarked.
- **Timing scope:** One shared environment and four sizes do not support performance guarantees. No end-to-end collection, memory, or concurrency experiment was performed.
- **Trust assumptions:** Authenticity, scan completeness, truthful time, and deployment identity are external. Full SLSA/CycloneDX conformance and cryptographic verification are not implemented.

- **Field evidence:** No real CI deployment, live Trivy experiment, participant study, or naturally occurring incident dataset was collected.

Broader evaluation should use authentic records from multiple real projects, independently assigned labels, stronger existing-policy baselines, and repeated measurements on controlled hardware. Claims of research novelty require a verified comparison with prior work.

7.2 Reproduction artifacts

The evaluated tool is version 0.1.0 from commit `fba5a6c`. The research folder contains complete case inputs, labels, rule outcomes, 400 raw timing observations, environment details, source hashes, test logs, editable table CSVs, and figure-generation scripts.

The synthetic policy assessment time was fixed at 09:00 UTC on 25 October 2026 for reproducibility. It is distinct from the actual run timestamp stored in the environment record. Regenerating timing data will change the measured values while decision labels should remain unchanged.

From the research folder, the core reproduction commands are:

```
python scripts/run_evaluation.py
python scripts/test_dashboard.py
python scripts/make_figures.py
python scripts/write_paper.py
```

The existing review manuscript and prototype source files were preserved. The present paper is a separate implementation study, and its conclusions are restricted to the reported experiment.

8. Conclusion

EvidenceGate makes release-evidence consistency explicit through ten policy checks. On two constructed suites, artifact binding rejected cases accepted by the ablation. Software and browser tests passed, and saved timing observations characterize local processing cost at four input sizes.

These findings support the implemented rule behavior and the value of separating identity checks from other policy conditions. They do not establish field effectiveness or safe software. Real evidence collection, independent evaluation, and authenticated records remain necessary for stronger research claims.